

LAPORAN PRATIKUM 1
PEMOGRAMAN BEROENTAS OBJEK



Disusun Oleh:
Silva Nurifvan
2511533004

Dosen Pengampu :
Nurfiah, S.ST, M.Kom
Asisten Praktikum :
Siti Fadhillah Rahmi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Pemrograman Berorientasi Objek yang berjudul “**Mini Bank**”.

Laporan ini dibuat sebagai salah satu bentuk pemenuhan tugas praktikum sekaligus sebagai media pembelajaran untuk memahami tahapan-tahapan yang dilakukan selama praktikum. Dalam laporan ini dibahas penyelesaian beberapa tugas, di antaranya penerapan fitur **tarik tunai**, pengelolaan beberapa akun dengan menggunakan **ArrayList**, serta fitur **ganti akun**.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan dan belum sepenuhnya sempurna. Oleh sebab itu, penulis sangat mengharapkan kritik dan saran yang membangun untuk perbaikan di masa yang akan datang. Penulis juga menyampaikan terima kasih kepada dosen pengampu, asisten praktikum, serta teman-teman yang telah memberikan bantuan dan dukungan selama proses penyusunan laporan ini.

Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca, khususnya dalam menambah wawasan dan memperdalam pemahaman mengenai pemrograman berorientasi objek menggunakan bahasa Java.

Padang, 12 September 2026

Silva Nurifvan

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek (PBO) merupakan pendekatan pemrograman yang menitikberatkan pada pengelompokan kode ke dalam objek-objek yang menggambarkan suatu entitas di dunia nyata. Beberapa konsep dasar dalam PBO, seperti **Class**, **Object**, **Constructor**, dan **Method**, dapat membantu dalam membangun perangkat lunak yang lebih terstruktur, modular, serta mudah dikembangkan. Dengan pendekatan tersebut, data dan fungsi yang berkaitan dapat digabungkan dalam sebuah class sehingga penggunaan kode menjadi lebih efisien dan pengelolaan data dapat dilakukan dengan lebih baik.

Penerapan Pemrograman Berorientasi Objek sangat sesuai untuk menggambarkan sistem perbankan karena berbagai entitas, seperti rekening dan nasabah, dapat direpresentasikan ke dalam bentuk **class**. Oleh karena itu, pada praktikum ini dibuat sebuah aplikasi “**Sistem Perbankan Mini**” berbasis **Java** sebagai simulasi sederhana dari sistem perbankan.

Aplikasi tersebut menerapkan beberapa aktivitas dasar dalam perbankan, seperti **setor tunai**, **tarik tunai**, dan **melihat informasi rekening**. Selain itu, program juga menggunakan struktur data **ArrayList** untuk menyimpan dan mengelola beberapa akun secara dinamis. Sistem ini juga dilengkapi dengan fitur **pergantian akun aktif**, sehingga pengguna dapat berpindah dan mengakses akun yang berbeda.

1.2 Tujuan

Adapun tujuan dari pelaksanaan praktikum ini adalah:

1. Memahami serta menerapkan konsep dasar Pemrograman Berorientasi Objek dalam membangun program simulasi sistem perbankan.
2. Menerapkan logika validasi pada berbagai transaksi agar data yang diproses sesuai dengan ketentuan program.
3. Menggunakan struktur **ArrayList** untuk menyimpan dan mengelola objek bertipe Rekening secara dinamis, serta menerapkan proses pencarian akun dalam program.

1.3 Manfaat

Manfaat yang diperoleh dari pelaksanaan praktikum ini antara lain:

1. Meningkatkan pemahaman praktis mahasiswa dalam mengubah dan merepresentasikan berbagai entitas di dunia nyata ke dalam bentuk objek pada program berorientasi objek.

2. Melatih kemampuan logika pemrograman, khususnya dalam melakukan pemeriksaan dan validasi input sehingga program dapat menangani data yang tidak sesuai tanpa mengalami kesalahan.
3. Memberikan pemahaman dasar mengenai proses pengelolaan data serta teknik pencarian elemen dalam **Collection** dengan memanfaatkan perulangan dan proses perbandingan.

BAB II

PEMBAHASAN

1.Langkah Kerja Praktik

1.1 Class Rekening

Pada class ini mendefinisikan atribut dan method dari objek rekening,

1.1.1 Kode Program

```
1 package pekani;
2
3 public class Rekening {
4     String nomorRekening;
5     String namaPemilik;
6     double saldo;
7
8     public Rekening(String nomor, String nama, double saldoAwal) {
9         nomorRekening = nomor;
10        namaPemilik = nama;
11        saldo = saldoAwal;
12        System.out.println("Rekening atas nama " + namaPemilik + " Berhasil dibuat dengan saldo Rp" + saldo);
13    }
14
15    public void setorTunai(double nominal) {
16        if (nominal > 0) {
17            saldo += nominal;
18            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini Rp" + saldo);
19        } else {
20            System.out.println("Gagal : Nominal setor harus lebih dari 0!");
21        }
22    }
23
24    public void tarikTunai(double nominal) {
25        if (nominal < 10000) {
26            System.out.println("Transaksi Gagal: Nominal minimal adalah Rp. 10.000");
27        } else if (nominal > saldo) {
28            System.out.println("Transaksi Gagal: Nominal penarikan tidak mencukupi saldo Rp. " + saldo);
29        } else if (saldo == 10000) {
30            System.out.println("Transaksi Gagal: Tidak bisa menarik karena anda sudah di batas saldo");
31        } else {
32            saldo -= nominal;
33            System.out.println("Sukses: Anda menarik saldo Rp " + nominal + " Sisa saldo = " + saldo);
34        }
35    }
36
37    public void cekInformasi() {
38        System.out.println("---- INFO REKENING ----");
39        System.out.println("No. Rekening : " + nomorRekening);
40        System.out.println("Nama Pemilik : " + namaPemilik);
41        System.out.println("Saldo Akhir : Rp" + saldo);
42        System.out.println("-----");
43    }
44 }
```

1.1.2 Uraian Kode Program

```
String nomorRekening;
String namaPemilik;
double saldo;
```

Pada bagian ini terdapat **constructor** yang berfungsi untuk menginisialisasi ketiga atribut menggunakan nilai yang dikirim dari Main, kemudian menampilkan konfirmasi pada output.

```
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini Rp" + saldo);
    } else {
        System.out.println("Gagal : Nominal setor harus lebih dari 0!");
    }
}
```

Method ini digunakan untuk menambahkan saldo dengan memeriksa terlebih dahulu apakah nominal yang dimasukkan lebih dari 0. Jika memenuhi syarat, saldo akan bertambah dan saldo terbaru ditampilkan. Namun, jika nominal kurang dari atau sama dengan 0, transaksi akan dinyatakan gagal karena nominal harus lebih dari 0.

```

public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal: Nominal minimal adalah Rp. 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Nominal penarikan tidak mencukupi saldo Rp. " + saldo);
    } else if (saldo == 10000) {
        System.out.println("Transaksi Gagal: Tidak bisa menarik karena anda sudah di batas saldo");
    } else {
        saldo -= nominal;
        System.out.println("Sukses: Anda menarik saldo Rp " + nominal + " Sisa saldo = " + saldo);
    }
}

```

Method ini digunakan untuk mengurangi saldo dengan melakukan beberapa pengecekan terlebih dahulu, yaitu apakah nominal kurang dari Rp10.000, nominal melebihi saldo, atau saldo sama dengan Rp10.000. Jika salah satu kondisi tersebut terpenuhi, proses penarikan akan dibatalkan dan menampilkan pesan penolakan yang sesuai. Jika semua ketentuan terpenuhi, saldo akan berkurang dan sistem menampilkan pesan bahwa penarikan berhasil beserta jumlah saldo yang tersisa.

```

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}

```

Method ini berfungsi untuk menampilkan informasi objek berupa nomor rekening, nama pemilik rekening, serta saldo akhir ke layar output.

1.2 Class Main

Class Main berisi method yang digunakan untuk membuat Scanner agar dapat menerima input dari pengguna, mendeklarasikan variabel objek, serta menjalankan perulangan while untuk menampilkan menu. Setiap pilihan pengguna kemudian diproses menggunakan struktur switch-case.

1.2.1 Kode Program

```
1 package pekan1;
2
3 import java.util.Scanner;
4 import java.util.ArrayList;
5
6 public class Main {
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         boolean isRunning = true;
10        ArrayList<Rekening> daftarRekening = new ArrayList<>();
11        Rekening akunAktif = null;
12
13        System.out.println("=== SISTEM PERBANKAN MINI ===");
14
15        while (isRunning) {
16            System.out.println("\nMenu Utama : ");
17            System.out.println("1. Buka rekening baru");
18            System.out.println("2. Setor tunai");
19            System.out.println("3. Tarik tunai");
20            System.out.println("4. Cek informasi rekening");
21            System.out.println("5. Ganti akun");
22            System.out.println("0. Keluar");
23            System.out.print("Pilih menu : ");
24
25            int pilihan = input.nextInt();
26            input.nextLine();
27
28            switch (pilihan) {
29                case 1:
30                    System.out.print("Masukkan No Rekening : ");
31                    String no = input.nextLine();
32                    System.out.print("Masukkan Nama Pemilik : ");
33                    String nama = input.nextLine();
34                    System.out.print("Masukkan Saldo Awal : ");
35                    double saldo = input.nextDouble();
36
37                    akunAktif = new Rekening(no, nama, saldo);
38                    daftarRekening.add(akunAktif);
39                    break;
40
41                case 2:
42                    if (akunAktif == null) {
43                        System.out.println("Error : Mohon maaf, anda belum memiliki nomor rekening!");
44                    } else {
45                        System.out.print("Masukkan nominal setor : ");
46                        double setor = input.nextDouble();
47                        akunAktif.setorTunai(setor);
48                    }
49                    break;
50
51                case 3:
52                    if (akunAktif == null) {
53                        System.out.println("Error : Mohon maaf, anda belum memiliki nomor rekening!");
54                    } else {
55                        System.out.print("Masukkan nominal tarik : ");
56                        double tarik = input.nextDouble();
57                        akunAktif.tarikTunai(tarik);
58                    }
59                    break;
60
61                case 4:
62                    if (akunAktif == null) {
63                        System.out.println("Error : Anda belum membuka rekening!");
64                    } else {
65                        akunAktif.cekInformasi();
66                    }
67                    break;
68
69                case 5:
70                    if (daftarRekening.isEmpty()) {
71                        System.out.println("Error : Belum ada rekening yang terdaftar!");
72                    } else {
73                        System.out.print("Masukkan No Rekening yang ingin diaktifkan : ");
74                        String cariNo = input.nextLine();
75                        Rekening ditemukan = null;
76
77                        for (Rekening r : daftarRekening) {
78                            if (r.nomorRekening.equals(cariNo)) {
79                                ditemukan = r;
80                                break;
81                            }
82                        }
83
84                        if (ditemukan == null) {
85                            System.out.println("Error : No Rekening " + cariNo + " tidak ditemukan!");
86                        } else {
87                            akunAktif = ditemukan;
88                            System.out.println("Berhasil!! Akun aktif sekarang: " + akunAktif.namaPemilik + " (" + akunAktif.nomorR
89                        }
90                    }
91                    break;
92
93                case 0:
94                    isRunning = false;
95                    System.out.println("Sistem ditutup. Terima Kasih!");
96                    break;
97
98                default:
99                    System.out.println("Pilihan tidak valid!");
100            }
101        }
102        input.close();
103    }
104 }
```

1.2.2 Uraian Kode Program

```
switch (pilihan) {  
    case 1:  
        System.out.print("Masukkan No Rekening : ");  
        String no = input.nextLine();  
        System.out.print("Masukkan Nama Pemilik : ");  
        String nama = input.nextLine();  
        System.out.print("Masukkan Saldo Awal : ");  
        double saldo = input.nextDouble();  
  
        akunAktif = new Rekening(no, nama, saldo);  
        daftarRekening.add(akunAktif);  
        break;
```

Menerima input berupa nomor rekening, nama pemilik, dan saldo awal, kemudian membuat objek baru berdasarkan data tersebut dan menyimpannya ke dalam variabel `akunAktif`.

```
case 2:  
    if (akunAktif == null) {  
        System.out.println("Error : Mohon maaf, anda belum memiliki nomor rekening!");  
    } else {  
        System.out.print("Masukkan nominal setor : ");  
        double setor = input.nextDouble();  
        akunAktif.setorTunai(setor);  
    }  
    break;
```

Memeriksa terlebih dahulu apakah `akunAktif` masih bernilai `null` sebelum meminta nominal dan menjalankan proses pada `akunAktif`, sehingga program tetap berjalan tanpa error ketika belum terdapat rekening.

```
case 3:  
    if (akunAktif == null) {  
        System.out.println("Error : Mohon maaf, anda belum memiliki nomor rekening!");  
    } else {  
        System.out.print("Masukkan nominal tarik : ");  
        double tarik = input.nextDouble();  
        akunAktif.tarikTunai(tarik);  
    }  
    break;
```

Mengecek terlebih dahulu apakah `akunAktif == null` untuk menampilkan pesan error. Jika `akun aktif` ada, program meminta input nominal penarikan via `input.nextDouble()` lalu memanggil method `akunAktif.tarikTunai` untuk memproses penarikan saldo.

```
case 4:  
    if (akunAktif == null) {  
        System.out.println("Error : Anda belum membuka rekening!");  
    } else {  
        akunAktif.cekInformasi();  
    }  
    break;
```

Sama seperti case 2, mengecek `akunAktif == null` lebih dulu, baru memanggil `akunAktif.cekInformasi()` untuk menampilkan data rekening yang sedang aktif.

```

case 5:
    if (daftarRekening.isEmpty()) {
        System.out.println("Error : Belum ada rekening yang terdaftar!");
    } else {
        System.out.print("Masukkan No Rekening yang ingin diaktifkan : ");
        String cariNo = input.nextLine();
        Rekening ditemukan = null;

        for (Rekening r : daftarRekening) {
            if (r.nomorRekening.equals(cariNo)) {
                ditemukan = r;
                break;
            }
        }
    }
}

```

Memeriksa apakah `daftarRekening.isEmpty()` untuk mencegah pencarian jika belum ada rekening yang dibuat. Jika ada, program meminta input nomor rekening dan menelusuri `daftarRekening` menggunakan perulangan serta perbandingan. Jika nomor rekening ditemukan, maka `akunAktif` diubah ke objek tersebut dan menampilkan konfirmasi sukses, jika tidak ditemukan, program menampilkan pesan error.

```

case 0:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima Kasih!");
    break;

default:
    System.out.println("Pilihan tidak valid!");
}

```

Mengubah `isRunning` menjadi `false` sehingga perulangan `while` berhenti, lalu mencetak pesan penutup.

2. Tugas

2.1 Implementasi Fitur Tarik Tunai

2.1.1 Kode Program

```

public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal: Nominal minimal adalah Rp. 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Nominal penarikan tidak mencukupi saldo Rp. " + saldo);
    } else if (saldo == 10000) {
        System.out.println("Transaksi Gagal: Tidak bisa menarik karena anda sudah di batas saldo");
    } else {
        saldo -= nominal;
        System.out.println("Sukses: Anda menarik saldo Rp " + nominal + " Sisa saldo = " + saldo);
    }
}

```

Transaksi akan ditolak apabila nominal yang dimasukkan kurang dari Rp10.000 atau melebihi saldo yang tersedia, sehingga saldo tidak dapat menjadi minus. Selain itu, jika saldo tepat Rp10.000, penarikan juga akan ditolak karena saldo tersebut dianggap sebagai batas minimum yang harus disisakan. Jika seluruh pengecekan telah terpenuhi, saldo akan dikurangi sesuai nominal penarikan dan hasil akhirnya ditampilkan pada output.

2.2 Multi Akun

2.2.1 Kode Program

```
ArrayList<Rekening> daftarRekening = new ArrayList<>();
Rekening akunAktif = null;

switch (pilihan) {
    case 1:
        System.out.print("Masukkan No Rekening : ");
        String no = input.nextLine();
        System.out.print("Masukkan Nama Pemilik : ");
        String nama = input.nextLine();
        System.out.print("Masukkan Saldo Awal : ");
        double saldo = input.nextDouble();

        akunAktif = new Rekening(no, nama, saldo);
        daftarRekening.add(akunAktif);
        break;

    case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Error : Belum ada rekening yang terdaftar!");
        } else {
            System.out.print("Masukkan No Rekening yang ingin diaktifkan : ");
            String cariNo = input.nextLine();
            Rekening ditemukan = null;

            for (Rekening r : daftarRekening) {
                if (r.nomorRekening.equals(cariNo)) {
                    ditemukan = r;
                    break;
                }
            }
        }
    }
```

ArrayList<Rekening> digunakan untuk menyimpan beberapa objek Rekening sekaligus dengan ukuran yang dapat bertambah secara otomatis setiap kali ada rekening baru yang ditambahkan. Setelah rekening berhasil dibuat pada case 1, objek tersebut dimasukkan ke dalam list sehingga data rekening sebelumnya tetap tersimpan. Sebelum pengguna diminta mencari rekening, program terlebih dahulu melakukan pengecekan untuk memastikan sudah ada rekening yang dibuat. Selanjutnya, nomor rekening pada setiap objek dibandingkan dengan input pengguna menggunakan equals() karena data yang dibandingkan berupa String. Jika rekening ditemukan, variabel akunAktif akan diarahkan ke objek rekening tersebut, sehingga transaksi berikutnya seperti setor saldo, tarik saldo, dan melihat informasi rekening dapat dilakukan pada rekening yang dipilih.

3.Output

1. Jika penarikan kurang dari Rp.10.000

```
=== SISTEM PERBANKAN MINI ===

Menu Utama :
1. Buka rekening baru
2. Setor tunai
3. Tarik tunai
4. Cek informasi rekening
5. Ganti akun
0. Keluar
Pilih menu : 1
Masukkan No Rekening : 0322796623
Masukkan Nama Pemilik : silva
Masukkan Saldo Awal : 20000
Rekening atas nama silva Berhasil dibuat dengan saldo Rp20000.0

Menu Utama :
1. Buka rekening baru
2. Setor tunai
3. Tarik tunai
4. Cek informasi rekening
5. Ganti akun
0. Keluar
Pilih menu : 3
Masukkan nominal tarik : 5000
Transaksi Gagal: Nominal minimal adalah Rp. 10.000
```

2. Jika penarikan melebihi saldo

```
Menu Utama :
1. Buka rekening baru
2. Setor tunai
3. Tarik tunai
4. Cek informasi rekening
5. Ganti akun
0. Keluar
Pilih menu : 3
Masukkan nominal tarik : 30000
Transaksi Gagal: Nominal penarikan tidak mencukupi saldo Rp. 20000.0
```

3. Membuka rekening dan mengganti akun

```
Menu Utama :
1. Buka rekening baru
2. Setor tunai
3. Tarik tunai
4. Cek informasi rekening
5. Ganti akun
0. Keluar
Pilih menu : 1
Masukkan No Rekening : 0322796623
Masukkan Nama Pemilik : silva
Masukkan Saldo Awal : 100000
Rekening atas nama silva Berhasil dibuat dengan saldo Rp100000.0

Menu Utama :
1. Buka rekening baru
2. Setor tunai
3. Tarik tunai
4. Cek informasi rekening
5. Ganti akun
0. Keluar
Pilih menu : 5
Masukkan No Rekening yang ingin diaktifkan : 0322796623
Berhasil!! Akun aktif sekarang: silva (0322796623)
```

BAB III

KESIMPULAN

Melalui praktikum ini dapat dipahami penerapan konsep pemrograman berorientasi objek secara langsung, yaitu pemisahan data dan perilaku melalui class Rekening, pembuatan objek menggunakan constructor, serta penerapan enkapsulasi pada logika validasi melalui method setorTunai dan tarikTunai. Selain itu, pengelolaan beberapa objek sekaligus dilakukan dengan menggunakan ArrayList sebagai pengganti satu variabel objek saja. Penambahan menu ganti akun juga melatih proses pencarian data di dalam ArrayList menggunakan perulangan serta perbandingan String dengan metode equals().